Individual Final Report

DATS 6303 - Deep Learning
Final Project - Multi-Modal Stock Prediction Using Time-Series and News Sentiment Fusion
Student: Mayur Patil

---

1. Introduction

Financial markets behave as complex, nonlinear systems influenced not only by numerical signals such as price trends and volatility but also by qualitative factors such as news sentiment, investor expectations, and real-time events. Traditional price-only forecasting models capture historical patterns but fail to incorporate the information shocks that frequently occur due to financial announcements, geopolitical events, and other sentiment-driven catalysts.

This project explores a multi-modal deep learning approach combining technical indicators, market index behaviour, and daily aggregated sentiment to predict next-day stock returns. Although the team collaborated conceptually, each member implemented their own version of the complete pipeline. This report documents my individual contribution, covering all six major components of the system, including:

1. Data engineering
2. Feature engineering
3. Sequential modelling and LSTM training
4. Sentiment extraction and integration
5. Fusion model development
6. Live next-day prediction system

My implementation is fully end-to-end and synthesizes both quantitative and qualitative information sources to improve predictive accuracy.

---

2. Description of My Individual Work

2.1 Data Engineering

I constructed the full data acquisition and preprocessing pipeline:

- Retrieved historical OHLCV (Open-High-Low-Close-Volume) data for the target stock.
- Retrieved daily price data for major market indices to use as macro features.
- Ensured date alignment across datasets and removed non-trading days.
- Cleaned missing values and normalized all features for model stability.

This step ensures that the downstream models receive consistent, high-quality numerical data.

2.2 Technical Feature Engineering

I engineered a comprehensive set of technical indicators that characterize daily market conditions.

Daily Return

Daily return describes relative change:

Return(t) = (Price(t) - Price(t-1)) / Price(t-1)

Moving Averages

A moving average smooths noise:

$MA_k(t)$ = (Price(t) + Price(t-1) + … + Price(t-(k-1))) / k

I computed 5, 10, and 20 day moving averages.

10-Day Volatility

Volatility measures the variability of daily returns:

$Volatility_{10}(t)$ = sqrt[average of squared differences between each of the last 10 returns and their mean]

Other Indicators

- Log returns
- Relative Strength Index (RSI)
- Market index returns (SPY, QQQ, DIA, etc.)
- Rolling standard deviations and rolling means

Table 1 - Technical Feature Summary

Category	Examples
Trend Indicators	Moving averages, log returns
Momentum Indicators	RSI, short-term return windows
Volatility Measures	Rolling standard deviation (10-day)
Market Context	Returns of SPY, QQQ, DIA, VIX-related

These features model price direction, strength, and instability.

2.3 Sequence Preparation

The predictive model uses a 30-day sliding window to create sequential samples.

How sequences are built

For each day t:

- Input sequence = features from day (t-29) to day t
- Target = return on day (t+1)

This creates input–output pairs appropriate for supervised learning.

Table 2 - Dataset Construction Summary

Item	Value
Total trading days	407
Window length	30 days
Usable sequences	378
Training split	70%
Validation split	15%
Test split	15%

My implementation ensures:

- No data leakage
- Strict chronological ordering
- Normalized feature tensors
- Efficient storage for training

2.4 LSTM Price Model Development

I developed and trained the recurrent neural network that extracts latent price representations from 30-day sequences.

Model behaviour

- Processes one day at a time
- Remembers patterns through hidden states
- Produces a final encoded vector summarizing recent market activity

This latent vector captures:

- Trend structure
- Volatility regimes

- Return clusters
- Medium-term momentum shifts

Training Procedure

I implemented:

- Mini-batch training loops
- Validation monitoring
- Early stopping to prevent overfitting
- Saving the best model weights

2.5 Sentiment Feature Engineering

I extracted and aggregated sentiment into three components per day:

1. Mean sentiment across articles
2. Sentiment variability (disagreement among articles)
3. Article count, representing volume of financial news

Sentiment vector form

Sentiment(t) = [mean sentiment, sentiment variation, count of articles]

To integrate sentiment with price sequences, I aligned sentiment dates with sequence end-dates and filled missing days with neutral defaults.

2.6 Fusion Model (Price LSTM + Sentiment MLP)

I designed and trained a fusion architecture that integrates both modalities.

Fusion Input

Fusion Input = [latent price vector; sentiment vector]

The combined vector is passed to a multilayer perceptron with dropout. The model outputs the predicted next-day return.

2.7 Live Prediction System

I implemented the deployed-style module that produces daily predictions:

1. Load the latest 30-day window
2. Compute the latent price representation
3. Extract sentiment for the current day
4. Feed combined input into the fusion model

5. Save forecast to timestamped output

Table 3 - Example Live Prediction

Symbol	Time Generated	Predicted Next-Day Return
AAPL	2025-12-07 14:35:12	+0.45%

This demonstrates real-time usefulness of the system.

---

4. Results

4.1 Evaluation Metrics

I evaluated performance using Mean Absolute Percentage Error (MAPE), Root Mean Square Error (RMSE), and Mean Absolute Error (MAE).

Table 4 - Model Evaluation Results

Dataset Split	MAPE (%)	RMSE	MAE
Training	4.09	27.83	23.37
Validation	4.90	32.66	31.02
Test	7.19	48.76	48.07

4.2 Price-Only vs. Fusion Model

Table 5 - Comparison of Models

Model Type	Test MAPE (%)
Price-Only Model	7.74
Price + Sentiment Model	7.19

Sentiment integration improved directional accuracy and reduced error during news-driven events.

---

5. Summary and Conclusions

My implementation of a multi-modal forecasting system demonstrates that combining price-based features with sentiment enhances predictive accuracy. The sequence model captures historical patterns, while sentiment accounts for news-driven deviations. The fusion model generalizes better than the price-only baseline and produces useful predictions in real time.

Key conclusions

- Price-only models miss important news-driven behaviour

- Sentiment improves generalization and trend recognition.
- The fusion of modalities yields the lowest error.
- Deployment-ready prediction is feasible.

Future work

- Use transformer-based textual sentiment (e.g., BERT embeddings).
- Add macroeconomic indicators.
- Train on multi-stock datasets.
- Introduce uncertainty estimation.

---

6. Percentage of Code Borrowed

Approximately 18% of my code comes from online references, mainly covering standard training loops, optimizer usage, and general deep learning boilerplate.

82% of the codebase was individually written, including:

- Feature generation
- Sequence construction
- LSTM model logic
- Sentiment engineering
- Fusion modeling
- Live prediction system

Final percentage: 18%

---

7. References

Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation.
Kingma, D., & Ba, J. (2014). *Adam Optimization Algorithm*.
Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
Araci, D. (2019). *FinBERT: Financial Sentiment Analysis Using Transformers*.
Murphy, J. (1999). *Technical Analysis of the Financial Markets*.
Achelis, S. (2001). *Technical Analysis from A to Z*.
Paszke, A., et al. (2019). *PyTorch: A Deep Learning Framework*.
Yahoo Finance API Documentation.
Pandas & NumPy Documentation.